Object-Oriented Programming

Java OOP — Complete Guide

1. The Basics

What is OOP?

OOP stands for Object-Oriented Programming. It is a style of writing code where you think about your program as a bunch of things (objects) instead of a long list of steps.

Imagine you are building a game. You do not write step 1, step 2, step 3 forever. Instead, you think: I have a Player, I have an Enemy, I have a Map. Each one is an object with its own data and its own actions.

Think of the real world. A phone is an object. It has properties (brand, color, battery level) and it can do things (call, take a photo, play music). In OOP, you model your code the exact same way.

OOP vs Procedural Programming

Before OOP was popular, people wrote code in a procedural style. Basically a recipe. You write function after function, top to bottom. That is fine for small stuff, but once your code grows it becomes a nightmare to read or change anything.

Procedural Style

```
String playerName = "Ali";
int playerHealth = 100;

void takeDamage(int damage) {
    playerHealth -= damage;
}
```

```
void printStatus() {
    System.out.println(playerName + " has " + playerHealth + " HP");
}
```

OOP Style

```
class Player {
    String name;
    int health;

    Player(String name) {
        this.name = name;
        this.health = 100;
    }

    void takeDamage(int damage) {
        this.health -= damage;
    }

    void printStatus() {
        System.out.println(name + " has " + health + " HP");
    }
}

Player p = new Player("Ali");
p.takeDamage(20);
p.printStatus();
```

In OOP the data and the functions that use that data live together inside the object. Much easier to manage when your program is big.

Procedural	OOP
Data and functions are separate	Data and functions are together inside objects
Hard to reuse code for similar things	Easy to reuse with inheritance
Gets messy as the program grows	Stays organized even in large projects

C is a classic example	Java, Python, C++ are examples

Core Concepts

Class

A class is like a blueprint. It does not actually exist as a thing in memory yet. It says: an object of this type will have these fields and these methods.

```
class Car {
    String color;
    int speed;
    String brand;

    void accelerate() {
        speed += 10;
        System.out.println("Speed is now: " + speed);
    }

    void brake() {
        speed -= 10;
        System.out.println("Speed is now: " + speed);
    }
}
```

Object

An object is a real instance created from the class. When you write new Car(), Java builds an actual car in memory. You can create as many objects as you want from the same class.

```
Car myCar = new Car();
myCar.color = "Red";
myCar.speed = 0;
myCar.accelerate();

Car friendCar = new Car();
friendCar.color = "Blue";
friendCar.speed = 0;
```

Instance

Instance is just another word for object. When you do new Car(), you are instantiating the Car class. Each instance has its own separate copy of the data.

```
Car car1 = new Car();
Car car2 = new Car();

car1.color = "Red";
car2.color = "Blue";

System.out.println(car1.color);
System.out.println(car2.color);
```

Attributes

Attributes are the variables inside a class. They hold the data that belongs to an object.

```
class Student {
    String name;
    int age;
    double gpa;
    String major;
}
```

Methods

Methods are the functions inside a class. They describe what the object can do and can read and change the object's attributes.

```
class Student {
    String name;
    double gpa;

    void study() {
        gpa += 0.1;
        System.out.println(name + " studied! GPA is now " + gpa);
    }

    void introduce() {
        System.out.println("Hi, I'm " + name + " and my GPA is " + gpa);
    }
```

```
    boolean isPassingWith(double minGpa) {
        return gpa >= minGpa;
    }
}

Student s = new Student();
s.name = "Sara";
s.gpa = 3.5;
s.study();
s.introduce();
System.out.println(s.isPassingWith(3.0));
```

Attributes are nouns — things the object HAS. Methods are verbs — things the object DOES.

2. Designing Classes

Class Syntax

The full skeleton of a class in Java:

```
class Person {
    String name;
    int age;
    String email;

    Person(String name, int age) {
        this.name = name;
        this.age  = age;
    }

    void greet() {
        System.out.println("Hello! My name is " + name);
    }

    void birthday() {
        age++;
```

```
        System.out.println("Happy birthday " + name + "! Now you are " + age);
    }

    String getSummary() {
        return name + ", age " + age;
    }
}
```

Constructor

A constructor is a special method that runs the moment you create an object. Its whole job is to set the object's starting values. You recognize it because it has the exact same name as the class and has no return type.

Default Constructor

If you do not write any constructor, Java quietly gives you an empty one. But you can write your own to set initial values.

```
class Player {
    String name;
    int health;
    int score;

    Player() {
        name   = "Unknown Player";
        health = 100;
        score  = 0;
        System.out.println("A new player was created with default values.");
    }
}

Player p = new Player();
System.out.println(p.name);
System.out.println(p.health);
```

Parameterized Constructor

You can pass values when creating the object so everything is set up with the right data from the start.

```
class Player {
    String name;
    int health;

    Player(String name, int health) {
        this.name   = name;
        this.health = health;
    }
}

Player p1 = new Player("Ali", 100);
Player p2 = new Player("Sara", 150);

System.out.println(p1.name + " - " + p1.health);
System.out.println(p2.name + " - " + p2.health);
```

Constructor Overloading

You can have more than one constructor in the same class as long as they take different parameters. Java picks the right one based on what you pass.

```
class Player {
    String name;
    int health;

    Player() {
        this("Guest", 100);
    }

    Player(String name) {
        this(name, 100);
    }

    Player(String name, int health) {
        this.name   = name;
        this.health = health;
    }
}

Player a = new Player();
```

```
Player b = new Player("Ali");
Player c = new Player("Sara", 200);
```

The this Keyword

this means the current object I am inside right now. You use it when a parameter has the same name as an attribute. this.name means the attribute, name alone means the parameter.

```
class Book {
    String title;
    String author;
    int pages;

    Book(String title, String author, int pages) {
        this.title  = title;
        this.author = author;
        this.pages  = pages;
    }

    void printInfo() {
        System.out.println(this.title + " by " + this.author + " (" + this.pages
+ " pages)");
    }
}

Book b = new Book("Clean Code", "Robert Martin", 431);
b.printInfo();
```

Accessing Attributes and Methods

After you have an object, you use the dot operator (.) to get to its attributes and call its methods.

```
class BankAccount {
    String owner;
    double balance;

    BankAccount(String owner, double balance) {
        this.owner   = owner;
```

```
        this.balance = balance;
    }

    void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited " + amount + ". New balance: " + balance);
    }

    void withdraw(double amount) {
        if (amount > balance) {
            System.out.println("Not enough money!");
        } else {
            balance -= amount;
            System.out.println("Withdrew " + amount + ". New balance: " +
balance);
        }
    }
}

BankAccount acc = new BankAccount("Omar", 500.0);
System.out.println(acc.owner);
acc.deposit(200);
acc.withdraw(100);
acc.withdraw(1000);
```

Destructor and Garbage Collection

In some languages like C++, you have to manually delete objects from memory. Java removes this headache with its Garbage Collector that automatically cleans up objects nothing points to anymore.

```
class HeavyObject {

    @Deprecated
    protected void finalize() {
        System.out.println("Object is being garbage collected");
    }
}
```

```
HeavyObject obj = new HeavyObject();
obj = null;
```

The static Keyword

A static attribute or method belongs to the class itself, not to any one object. All objects share the same static variable.

```
class Counter {
    static int count = 0;
    int id;

    Counter() {
        count++;
        id = count;
    }

    static void showTotal() {
        System.out.println("Total counters created: " + count);
    }
}

Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();

Counter.showTotal();
System.out.println(c1.id);
System.out.println(c2.id);
```

Instance	Static
Belongs to each object separately	Belongs to the class, shared by all objects
Each object has its own copy	One copy total, no matter how many objects exist
Called via object: obj.method()	Called via class: ClassName.method()

The toString() Method

Every class in Java automatically has a toString() method. By default it returns something ugly like Car@3d04a311. Override it to return something useful.

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age  = age;
    }

    @Override
    public String toString() {
        return "Person { name=" + name + ", age=" + age + " }";
    }
}

Person p = new Person("Nour", 22);
System.out.println(p);
System.out.println("" + p);
```

3. Encapsulation

What is Encapsulation?

Encapsulation means you hide the insides of an object and only let the outside world interact through a controlled door.

Think about a bank ATM. You put in your card and PIN, press a button, and money comes out. You have no idea what happens inside. The complexity is hidden.

In code, encapsulation means making attributes private and providing public methods to access or change them safely.

Access Modifiers

Modifier	Who can access it?
public	Everyone, from anywhere
private	Only inside the same class
protected	Same class + subclasses + same package
(no modifier)	Same package only

```
class Person {
    public    String name;
    private   int    age;
    protected String email;
}

Person p = new Person();
p.name = "Ali";
p.age  = 25;
```

Getters and Setters

Since private attributes cannot be touched from outside, you provide getter and setter methods to read and update them. The big advantage is you can add validation logic inside the setter.

```
class Person {
    private String name;
    private int    age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name == null || name.isEmpty()) {
            System.out.println("Name can't be empty!");
```

```
            return;
        }
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age < 0 || age > 150) {
            System.out.println("That's not a valid age: " + age);
            return;
        }
        this.age = age;
    }
}

Person p = new Person();
p.setName("Layla");
p.setAge(25);
p.setAge(-5);

System.out.println(p.getName());
System.out.println(p.getAge());
```

Full Encapsulation Example

```
class BankAccount {
    private String owner;
    private double balance;

    public BankAccount(String owner, double initialBalance) {
        this.owner   = owner;
        this.balance = initialBalance >= 0 ? initialBalance : 0;
    }

    public String getOwner()   { return owner; }
```

```
    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("Deposit amount must be positive.");
            return;
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Withdrawal amount must be positive.");
            return;
        }
        if (amount > balance) {
            System.out.println("Insufficient funds.");
            return;
        }
        balance -= amount;
    }

    @Override
    public String toString() {
        return owner + "'s account: $" + balance;
    }
}

BankAccount acc = new BankAccount("Ahmed", 1000);
acc.deposit(500);
acc.withdraw(200);
acc.withdraw(5000);
System.out.println(acc);
```

4. Inheritance

What is Inheritance?

Inheritance lets a class take everything from another class and then add or change things on top. The class being taken from is called the parent (superclass). The class that takes is called the child (subclass).

If Dog and Cat both have a name and can eat, you write that once in Animal and both inherit it. No duplicate code.

```
class Animal {
    String name;
    int age;

    Animal(String name, int age) {
        this.name = name;
        this.age  = age;
    }

    void eat() {
        System.out.println(name + " is eating.");
    }

    void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

class Dog extends Animal {
    String breed;

    Dog(String name, int age, String breed) {
        super(name, age);
        this.breed = breed;
    }

    void bark() {
        System.out.println(name + " says: Woof!");
    }
}
```

```
Dog d = new Dog("Max", 3, "Labrador");
d.eat();
d.sleep();
d.bark();
System.out.println(d.breed);
```

Types of Inheritance

Single Inheritance

One child inherits from one parent.

```
class Vehicle {
    int speed;
    void move() { System.out.println("Vehicle is moving at " + speed); }
}

class Car extends Vehicle {
    String brand;
    void honk() { System.out.println(brand + " goes beep!"); }
}

Car c = new Car();
c.speed = 80;
c.brand = "Toyota";
c.move();
c.honk();
```

Multilevel Inheritance

A chain — each level inherits from the one above it and adds more behavior.

```
class Animal {
    void breathe() { System.out.println("Breathing..."); }
}

class Mammal extends Animal {
    void feedMilk() { System.out.println("Feeding milk to young..."); }
}
```

```
class Dog extends Mammal {
    void bark() { System.out.println("Woof!"); }
}

Dog d = new Dog();
d.breathe();
d.feedMilk();
d.bark();
```

Hierarchical Inheritance

Multiple children inherit from the same one parent.

```
class Shape {
    String color;
    void describe() { System.out.println("I am a " + color + " shape."); }
}

class Circle extends Shape {
    double radius;
    double area() { return 3.14 * radius * radius; }
}

class Rectangle extends Shape {
    double width, height;
    double area() { return width * height; }
}

class Triangle extends Shape {
    double base, height;
    double area() { return 0.5 * base * height; }
}

Circle c = new Circle();
c.color  = "Red";
c.radius = 5;
c.describe();
System.out.println(c.area());
```

Multiple Inheritance via Interfaces

Java does not let a class extend two classes at once. But you can implement multiple interfaces to get something similar.

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
    public void fly()  { System.out.println("Duck is flying!"); }
    public void swim() { System.out.println("Duck is swimming!"); }
}

Duck d = new Duck();
d.fly();
d.swim();
```

Method Overriding

When a child class writes a method with the exact same name and parameters as the parent, the child's version replaces the parent's version for that object.

```
class Animal {
    String name;

    Animal(String name) { this.name = name; }

    void speak() {
        System.out.println(name + " makes a sound.");
    }
}

class Dog extends Animal {
    Dog(String name) { super(name); }
```

```
    @Override
    void speak() {
        System.out.println(name + " says: Woof!");
    }
}

class Cat extends Animal {
    Cat(String name) { super(name); }

    @Override
    void speak() {
        System.out.println(name + " says: Meow!");
    }
}

Animal a = new Animal("Creature");
Dog   d  = new Dog("Rex");
Cat   c  = new Cat("Luna");

a.speak();
d.speak();
c.speak();
```

The super Keyword

super is like this but for the parent class. Use it to call the parent's constructor or the parent's version of a method.

```
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
        System.out.println("Animal created: " + name);
    }

    void describe() {
        System.out.println("I am an animal named " + name);
```

```
}

class Dog extends Animal {
    String breed;

    Dog(String name, String breed) {
        super(name);
        this.breed = breed;
        System.out.println("Dog created with breed: " + breed);
    }

    @Override
    void describe() {
        super.describe();
        System.out.println("I am also a dog of breed: " + breed);
    }
}

Dog d = new Dog("Buddy", "Beagle");
d.describe();
```

super() must be the first line in the child constructor if you write it explicitly. If you do not write it, Java adds it automatically.

The final Keyword

final means you cannot change or extend this. It works on variables, methods, and classes.

```
final double PI = 3.14159;
PI = 3.0;

class Animal {
    final void breathe() {
        System.out.println("Breathing air.");
    }
}

class Dog extends Animal {
```

```
}

final class ImmutableClass { }
```

5. Polymorphism

What is Polymorphism?

Polymorphism means many forms. In code, it means the same method name can do different things depending on what object or what parameters you use it with. It lets you write general code that works correctly for many different types.

Compile-time Polymorphism — Method Overloading

Overloading means having multiple methods with the same name but different parameters. Java picks which one to call based on the arguments at compile time.

```
class Printer {
    void print(String text) {
        System.out.println("String: " + text);
    }

    void print(int number) {
        System.out.println("Number: " + number);
    }

    void print(String text, int times) {
        for (int i = 0; i < times; i++) {
            System.out.println(text);
        }
    }

    void print(double value) {
        System.out.printf("Double: %.2f%n", value);
    }
}
```

```
Printer p = new Printer();
p.print("Hello");
p.print(42);
p.print("Hi", 3);
p.print(3.14159);
```

```
class Calculator {
    int    add(int a, int b)          { return a + b; }
    int    add(int a, int b, int c)   { return a + b + c; }
    double add(double a, double b)    { return a + b; }
    String add(String a, String b)    { return a + b; }
}

Calculator calc = new Calculator();
System.out.println(calc.add(2, 3));
System.out.println(calc.add(1, 2, 3));
System.out.println(calc.add(1.5, 2.5));
System.out.println(calc.add("Hello", " World"));
```

Runtime Polymorphism — Method Overriding

This is when a parent class variable holds a child class object, and the child's overridden method runs instead of the parent's — decided at runtime, not compile time.

```
class Animal {
    void speak() { System.out.println("Some generic sound"); }
}

class Dog extends Animal {
    @Override
    void speak() { System.out.println("Woof!"); }
}

class Cat extends Animal {
    @Override
    void speak() { System.out.println("Meow!"); }
}
```

```
class Cow extends Animal {
    @Override
    void speak() { System.out.println("Moo!"); }
}

Animal a1 = new Dog();
Animal a2 = new Cat();
Animal a3 = new Cow();

a1.speak();
a2.speak();
a3.speak();
```

Very useful when you loop over a list of mixed types:

```
Animal[] farm = { new Dog(), new Cat(), new Cow(), new Dog() };

for (Animal animal : farm) {
    animal.speak();
}
```

This is called dynamic dispatch. Java automatically calls the right method for the actual object type, even when you are using a parent type variable.

Operator Overloading

Operator overloading means giving a custom meaning to operators like + or * for your own classes. Java does NOT support this for custom classes. Languages like C++ and Python do. In Java you write a regular method instead.

```
class Vector {
    double x, y;

    Vector(double x, double y) {
        this.x = x;
        this.y = y;
    }

    Vector add(Vector other) {
        return new Vector(this.x + other.x, this.y + other.y);
```

```
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}

Vector v1 = new Vector(1, 2);
Vector v2 = new Vector(3, 4);
Vector v3 = v1.add(v2);
System.out.println(v3);
```

6. Abstraction

What is Abstraction?

Abstraction means you show only what is needed and hide the rest. You focus on what something does, not how it does it.

Think about a coffee machine. You press make coffee and coffee comes out. You do not know how the water heats or how the pump works. All that complexity is hidden behind a simple button.

In Java, abstraction is done through abstract classes and interfaces.

Abstract Classes

An abstract class cannot be instantiated — you cannot do new AbstractClass(). It is a base for other classes. It can have both regular implemented methods and abstract unimplemented methods.

```
abstract class Shape {
    String color;

    Shape(String color) {
        this.color = color;
```

```
    }

    abstract double area();
    abstract double perimeter();

    void describe() {
        System.out.println("I am a " + color + " shape.");
        System.out.println("Area: " + area());
        System.out.println("Perimeter: " + perimeter());
    }
}

class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    double area() { return Math.PI * radius * radius; }

    @Override
    double perimeter() { return 2 * Math.PI * radius; }
}

class Rectangle extends Shape {
    double width, height;

    Rectangle(String color, double width, double height) {
        super(color);
        this.width  = width;
        this.height = height;
    }

    @Override
    double area() { return width * height; }
```

```
    @Override
    double perimeter() { return 2 * (width + height); }
}

Circle c = new Circle("Green", 5);
c.describe();

Rectangle r = new Rectangle("Blue", 4, 6);
r.describe();
```

Interfaces

An interface is like a contract. Any class that signs it MUST have these methods. The interface does not say how — just that they must exist. A class can implement multiple interfaces.

```
interface Drawable {
    void draw();
}

interface Resizable {
    void resize(double factor);
}

interface Saveable {
    void save(String filename);
}

class Canvas implements Drawable, Resizable, Saveable {

    @Override
    public void draw() {
        System.out.println("Drawing on canvas...");
    }

    @Override
    public void resize(double factor) {
        System.out.println("Resizing canvas by factor: " + factor);
    }
```

```
    @Override
    public void save(String filename) {
        System.out.println("Saving canvas to: " + filename);
    }
}

Canvas c = new Canvas();
c.draw();
c.resize(1.5);
c.save("art.png");
```

Default Methods in Interfaces (Java 8+)

Since Java 8, interfaces can have default methods with a body. This lets you add new methods without breaking every class that implements it.

```
interface Logger {
    void log(String message);

    default void logInfo(String message) {
        log("[INFO] " + message);
    }

    default void logError(String message) {
        log("[ERROR] " + message);
    }
}

class ConsoleLogger implements Logger {
    @Override
    public void log(String message) {
        System.out.println(message);
    }
}

ConsoleLogger logger = new ConsoleLogger();
logger.log("raw message");
logger.logInfo("Server started");
logger.logError("Disk full!");
```

Abstract Class vs Interface

Feature	Abstract Class	Interface
Can have instance variables?	Yes	No (only constants)
Can have constructors?	Yes	No
Can have implemented methods?	Yes	Only with default
How many can a class use?	Only one (extends)	Many (implements)
Use when...	Related classes share common state or code	Unrelated classes share behavior

```
abstract class Employee {
    String name;
    double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    abstract double calculateBonus();

    double totalPay() {
        return baseSalary + calculateBonus();
    }
}

class Manager extends Employee {
    Manager(String name, double baseSalary) {
        super(name, baseSalary);
    }
```

```
    @Override
    double calculateBonus() { return baseSalary * 0.3; }
}

class Developer extends Employee {
    Developer(String name, double baseSalary) {
        super(name, baseSalary);
    }

    @Override
    double calculateBonus() { return baseSalary * 0.2; }
}

Manager   mgr = new Manager("Hana", 8000);
Developer dev = new Developer("Karim", 6000);

System.out.println(mgr.name + " earns: " + mgr.totalPay());
System.out.println(dev.name + " earns: " + dev.totalPay());
```

Use an abstract class when the things share a lot of code and genuinely are the same kind of thing. Use an interface when different unrelated things just need to promise they can do a certain action.

7. Putting It All Together

Full Example Using All Four Pillars

A small game character system that uses encapsulation, inheritance, polymorphism, and abstraction together.

```
abstract class GameCharacter {
    private String name;
    private int    health;

    GameCharacter(String name, int health) {
```

```
        this.name   = name;
        this.health = health;
    }

    public String getName()   { return name; }
    public int    getHealth() { return health; }

    public void takeDamage(int damage) {
        health = Math.max(0, health - damage);
    }

    public boolean isAlive() { return health > 0; }

    abstract int attack();

    @Override
    public String toString() {
        return name + " [HP: " + health + "]";
    }
}

class Warrior extends GameCharacter {
    private int strength;

    Warrior(String name, int strength) {
        super(name, 150);
        this.strength = strength;
    }

    @Override
    public int attack() {
        return strength * 2;
    }
}

class Mage extends GameCharacter {
    private int mana;
```

```
    Mage(String name, int mana) {
        super(name, 80);
        this.mana = mana;
    }

    @Override
    public int attack() {
        return mana * 3;
    }
}

class Archer extends GameCharacter {
    private int agility;

    Archer(String name, int agility) {
        super(name, 100);
        this.agility = agility;
    }

    @Override
    public int attack() {
        return agility * 2 + 10;
    }
}

GameCharacter[] party = {
    new Warrior("Thor",   25),
    new Mage("Merlin",    30),
    new Archer("Legolas", 20)
};

GameCharacter boss = new Warrior("Dark Lord", 40);

System.out.println("Party attacks the boss!\n");

for (GameCharacter hero : party) {
    int dmg = hero.attack();
    boss.takeDamage(dmg);
```

```
    System.out.println(hero.getName() + " deals " + dmg + " damage. Boss: " +
boss);
}

System.out.println("\nBoss alive? " + boss.isAlive());
```